TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 2: Algorithm Analysis and Asymptotic Notation

Foreign Trade University

Contents

1 Introduction

In Week 1 we used Big-O notation informally to describe how the running time of Gale–Shapley scales with n. This week we make that vocabulary precise. Following Kleinberg & Tardos, Chapter 2, we will:

1. Define O, Ω, and Θ rigorously, with the formal "there exist constants c, n_0" definitions, and prove basic properties (transitivity, sums, products).
2. Build a mental hierarchy of common growth rates and learn to recognize which class an algorithm's running time falls into just by reading its code (counting loop iterations, recursive calls, etc.).
3. Discuss *polynomial time* as the standard (if imperfect) notion of "efficient", and contrast it with exponential-time brute force.
4. Work through a full worked example – **binary search** – analyzing its running time via a recurrence relation, both directly (the iterative version) and via the recursion tree (the recursive version).

This week's practical exercises (17 problems) ask you to implement and empirically verify each of these ideas: counting operations in loops, writing "witness checkers" for O, Ω, Θ, solving small recurrences by direct simulation, and analyzing classic algorithms (binary search, heaps, dynamic arrays, Karatsuba-style divide and conquer, and a more careful re-implementation of Gale–Shapley using arrays instead of dictionaries).

2 Asymptotic Notation, Formally

2.1 Big-O: an asymptotic upper bound

Definition 2.1 (Big-O)**.** *Let $f, g : \mathbb{N} \to \mathbb{R}^+$. We say $f(n) = O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that*

$$f(n) \leq c \cdot g(n) \qquad \text{for all } n \geq n_0.$$

Informally: for sufficiently large n, f grows no faster than a constant multiple of g.

Example 2.2. *$f(n) = 3n^2 + 100n + 7$ satisfies $f(n) = O(n^2)$. Take $c = 4$: for $n \geq n_0 = 105$,*

$$3n^2 + 100n + 7 \leq 3n^2 + n^2 + n^2 = 4n^2,$$

because once $n \geq 105$, $100n + 7 \leq n^2$ (in fact $100n \leq n^2$ already holds for $n \geq 100$, and the $+7$ is absorbed by the slack). The exact choice of n_0 does not matter – only that some *valid pair (c, n_0) exists.*

A common point of confusion: $O(g(n))$ is an *upper bound* on f, but it does not have to be *tight*. Every function that is $O(n)$ is automatically also $O(n^2)$, $O(n^3)$, etc. (Problem 3 in this week's exercises asks you to implement witness checkers that make this concrete: given proposed (c, n_0), verify $f(n) \leq c\, g(n)$ on a sample of n values.)

2.2 Big-Omega: an asymptotic lower bound

Definition 2.3 (Big-Omega)**.** *$f(n) = \Omega(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that*

$$f(n) \geq c \cdot g(n) \qquad \text{for all } n \geq n_0.$$

Informally: for sufficiently large n, f grows at least as fast as a constant multiple of g.

Big-Omega is the mirror image of Big-O: $f(n) = \Omega(g(n))$ if and only if $g(n) = O(f(n))$.

2.3 Big-Theta: tight bounds

Definition 2.4 (Big-Theta)**.** *$f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$* and *$f(n) = \Omega(g(n))$. Equivalently, there exist constants $c_1, c_2 > 0$ and n_0 such that*

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \qquad \text{for all } n \geq n_0.$$

When we say "algorithm A runs in time $\Theta(n \log n)$", we mean the running time is *sandwiched* between two constant multiples of $n \log n$ – this is the strongest and most informative of the three notations, and the one we aim for whenever possible.

2.4 Basic properties

Proposition 2.5 (Transitivity)**.** *If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$. The analogous statements hold for Ω and Θ.*

Proof. By assumption there exist c_1, n_1 with $f(n) \leq c_1 g(n)$ for $n \geq n_1$, and c_2, n_2 with $g(n) \leq c_2 h(n)$ for $n \geq n_2$. Let $n_0 = \max(n_1, n_2)$ and $c = c_1 c_2$. For $n \geq n_0$, $f(n) \leq c_1 g(n) \leq c_1 c_2 h(n) = c\, h(n)$. □

Proposition 2.6 (Sums)**.** *If $f_1(n) = O(g(n))$ and $f_2(n) = O(g(n))$, then $(f_1 + f_2)(n) = O(g(n))$. More generally, if $f(n) = O(g(n))$ and $h(n) = O(g(n))$, then $\max(f(n), h(n)) = O(g(n))$ – so the sum of a constant number of terms is dominated by the asymptotically largest term, with the others absorbed into the constant.*

Proposition 2.7 (Polynomials)**.** *For a polynomial $p(n) = a_d n^d + a_{d-1} n^{d-1} + \cdots + a_0$ with $a_d > 0$, $p(n) = \Theta(n^d)$.*

Proof sketch. For the upper bound, $p(n) \leq (|a_d| + |a_{d-1}| + \cdots + |a_0|) \cdot n^d$ for $n \geq 1$ (since $n^i \leq n^d$ for $i \leq d$ and $n \geq 1$). For the lower bound, for n large enough the leading term $a_d n^d$ dominates the sum of the absolute values of the remaining terms, so $p(n) \geq \frac{a_d}{2} n^d$. □

3 The Hierarchy of Growth Rates

A useful ordering of common growth rates, from slowest- to fastest-growing (each is asymptotically dominated by the next, i.e. $o(\cdot)$, a *strict* version of O):

$$O(1) \subset O(\log n) \subset O(\sqrt{n}) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!).$$

3.1 Reading growth rates off code

The most common way a growth rate arises is by counting loop iterations or recursive calls.

- A single loop `for i in range(n)` with $O(1)$ work per iteration: $\Theta(n)$.
- A nested loop `for i in range(n):  for j in range(n)`: $\Theta(n^2)$ (Problem 2).
- A *triangular* nested loop `for i in range(n):  for j in range(i)`: the inner loop runs $0 + 1 + \cdots + (n-1) = \binom{n}{2} = \Theta(n^2)$ times total – same asymptotic class as the full nested loop, just with a smaller constant (Problem 2).
- A loop that halves its range each iteration, `while n > 1:  n = n // 2`: $\Theta(\log n)$ iterations (Problem 5).

Growth rate	Name	Typical example
$\Theta(1)$	constant	array index access
$\Theta(\log n)$	logarithmic	binary search
$\Theta(\sqrt{n})$	root	some number-theoretic primality checks
$\Theta(n)$	linear	scanning a list, finding max
$\Theta(n \log n)$	linearithmic	merge sort, heapsort, sorting with a heap
$\Theta(n^2)$	quadratic	insertion sort, naive max/min via two scans is $\Theta(n)$, nested-loop pairwise comparisc
$\Theta(n^3)$	cubic	naive matrix multiplication, naive triple-nested loops
$\Theta(2^n)$	exponential	brute-force subset enumeration, naive recursive Fibonacci
$\Theta(n!)$	factorial	brute-force enumeration of all permutations (e.g. Week 1 Problem 5)

Table 1: Common running-time classes, slowest to fastest growing.

3.2 Recurrences

Many algorithms (especially recursive ones) have running times described by a **recurrence relation**: an equation for $T(n)$ in terms of T at smaller arguments. Three recurrences recur throughout this course:

- $T(n) = T(n-1) + O(1)$, $T(0) = O(1) \Rightarrow T(n) = \Theta(n)$ (linear recursion – e.g. a simple countdown, Problem 4).
- $T(n) = T(n/2) + O(1)$, $T(1) = O(1) \Rightarrow T(n) = \Theta(\log n)$ (halving recursion – e.g. binary search, Problem 5).
- $T(n) = 2T(n/2) + O(n)$, $T(1) = O(1) \Rightarrow T(n) = \Theta(n \log n)$ (divide-and-conquer with linear merge – e.g. mergesort, Problem 6).

Example 3.1 (Solving $T(n) = T(n/2) + 1$)**.** *Assume $n = 2^k$. Unrolling: $T(2^k) = T(2^{k-1}) + 1 = T(2^{k-2}) + 2 = \cdots = T(2^0) + k = T(1) + k$. Since $k = \log_2 n$, we get $T(n) = T(1) + \log_2 n = \Theta(\log n)$.*

Example 3.2 (Solving $T(n) = 2T(n/2) + n$)**.** *Again assume $n = 2^k$. At recursion depth i, there are 2^i sub-problems, each of size $n/2^i$, and each level contributes $2^i \cdot (n/2^i) = n$ total work (for the "$+n$" combine step, split proportionally across the 2^i sub-calls). There are $k = \log_2 n$ levels (depth 0 through $k-1$, plus base cases), so the total work is $n \cdot \log_2 n + \Theta(n) = \Theta(n \log n)$.*

Problems 4–6 ask you to implement each of these three recursions *exactly as written* (so the number of recursive calls matches the formula) and verify the closed-form solutions numerically for a range of n.

4 Polynomial Time and Tractability

Definition 4.1 (Polynomial-time algorithm)**.** *An algorithm runs in **polynomial time** if its worst-case running time is $O(n^k)$ for some constant k that does not depend on the input.*

Kleinberg & Tardos propose polynomial time as a useful (if imperfect) proxy for "efficient", for two main reasons:

1. **Closure under composition.** If you call a polynomial-time subroutine a polynomial number of times, the overall algorithm is still polynomial-time (a polynomial composed with a polynomial is a polynomial). This makes polynomial-time algorithms *compositional* – you can build complex polynomial-time algorithms out of simpler polynomial-time pieces.

2. **Sharp practical divide.** In practice, there is often a genuine qualitative jump between polynomial algorithms (which scale to large n) and exponential algorithms (which become infeasible for even moderately-sized n). An $O(n^3)$ algorithm on $n = 1000$ does 10^9 operations – seconds on a modern machine. A $O(2^n)$ algorithm on $n = 40$ already does over 10^{12} operations.

Running time	$n = 10$	$n = 30$	$n = 50$
n	10	30	50
$n \log n$	≈ 33	≈ 147	≈ 282
n^2	100	900	2,500
n^3	1,000	27,000	125,000
2^n	1,024	1,073,741,824	$\approx 1.1 \times 10^{15}$
$n!$	3,628,800	$\approx 2.65 \times 10^{32}$	$\approx 3.0 \times 10^{64}$

Table 2: Growth of running time with n, for fixed operation counts – the qualitative divide between polynomial and super-polynomial growth becomes stark by $n = 30$.

This is precisely why brute-force enumeration of all $n!$ matchings (Week 1, Problem 5) is fine for $n \leq 6$ but utterly infeasible for $n = 50$, while Gale–Shapley's $O(n^2)$ bound scales gracefully.

4.1 Caveats

Polynomial time is a useful *coarse* filter, but it is not the whole story:

- An $O(n^{100})$ algorithm is polynomial but useless in practice.
- An $O(2^{n/1000})$ algorithm is exponential but might be fine for the n's that actually arise.
- Constant factors matter in practice (Theory Question 4 / Week 1 revisited): asymptotics are about *scaling*, not absolute speed at any fixed n.

Nonetheless, "is there a polynomial-time algorithm for this problem?" remains one of the most important questions an algorithm designer asks, and sets up the contrast (later in the course) with NP-hard problems for which no polynomial-time algorithm is known.

5 Worked Example: Binary Search

Binary search is this week's running example because it cleanly illustrates almost every idea above: a simple loop invariant, a $\Theta(\log n)$ recurrence, and both an iterative and a recursive implementation that must be shown equivalent.

5.1 Problem statement

Given a sorted array $A[0..n-1]$ and a target value x, find an index i such that $A[i] = x$, or report that no such index exists.

5.2 Iterative algorithm and loop invariant

Algorithm 1 BinarySearchIterative(A, x)

```
1: lo ← 0, hi ← n − 1
2: while lo ≤ hi do
3:     mid ← ⌊(lo + hi)/2⌋
4:     if A[mid] = x then
5:         return mid
6:     else if A[mid] < x then
7:         lo ← mid + 1
8:     else
9:         hi ← mid − 1
10:    end if
11: end while
12: return −1
```

Proposition 5.1 (Correctness via loop invariant)**.** *At the start of every iteration,* ***if*** x ***is present in*** A***, its index lies in*** $[lo, hi]$**.**

Proof. Initialization. Before the first iteration, $[lo, hi] = [0, n-1]$, the entire array, so the invariant holds trivially.

Maintenance. Suppose the invariant holds at the start of an iteration and $x \neq A[mid]$. Since A is sorted: if $x > A[mid]$, then x (if present) must be at an index $> mid$, so updating $lo \leftarrow mid + 1$ preserves the invariant. Symmetrically if $x < A[mid]$, updating $hi \leftarrow mid - 1$ preserves the invariant.

Termination. The loop ends either by finding x (return mid, correct by the check), or when $lo > hi$, meaning the invariant's interval $[lo, hi]$ is empty – by the invariant, x cannot be present, so returning -1 is correct. □

5.3 Running-time analysis

Each iteration either terminates or shrinks $hi - lo + 1$ (the size of the candidate range) to *at most half* its previous value (since mid splits $[lo, hi]$ roughly in half, and one half is discarded). Starting from a range of size n, after k iterations the range has size at most $\lceil n/2^k \rceil$. The loop must terminate once the range size reaches 0, i.e. once $k > \log_2 n$. Hence the iterative algorithm performs $\Theta(\log n)$ iterations, each doing $O(1)$ work, for a total running time of $\Theta(\log n)$.

5.4 Recursive version and recursion tree

This satisfies the recurrence $T(n) = T(n/2) + O(1)$ from Section 3.2, whose solution is $T(n) = \Theta(\log n)$ – matching the iterative analysis. The recursion "tree" here is actually a single *path* (each call makes exactly one recursive call), of depth $\lceil \log_2(n+1) \rceil$. Problem 7 asks you to implement both versions and to compute this depth directly; Problem 8 extends binary search to find the first/last occurrence of a repeated value (still $O(\log n)$ per query), and Problem 9 applies the same divide-and-decide idea to searching a *rotated* sorted array.

Algorithm 2 BinarySearchRecursive(A, x, lo, hi)

```
if lo > hi then
    return −1
end if
mid ← ⌊(lo + hi)/2⌋
if A[mid] = x then
    return mid
else if A[mid] < x then
    return BinarySearchRecursive(A, x, mid + 1, hi)
else
    return BinarySearchRecursive(A, x, lo, mid − 1)
end if
```

6 Other Examples of Algorithm Analysis

6.1 Finding the maximum and minimum: a tighter constant

Finding the maximum of n elements takes $n-1$ comparisons (one linear scan). Finding both the max *and* the min naively takes $2(n-1)$ comparisons (two scans) – still $\Theta(n)$, but with a worse constant. A classic trick (Problem 10) processes elements in *pairs*: for each pair, one comparison determines the local max/min candidate, then two more comparisons update the running max and min – 3 comparisons per pair instead of 4, giving roughly $\frac{3n}{2}$ comparisons total. Both algorithms are $\Theta(n)$, but this is a good illustration that *constants matter* even within the same Θ class, and that careful counting can reveal real (if modest) improvements.

6.2 Selection: median via sorting vs. quickselect

Finding the median of n unsorted elements by sorting first costs $\Theta(n \log n)$. **Quickselect** (Problem 11) finds the k-th smallest element directly, using a divide-and-conquer partition step (as in quicksort) but recursing into only *one* side: $T(n) = T(\alpha n) + O(n)$ for some $\alpha < 1$ in expectation, giving $\mathbb{E}[T(n)] = \Theta(n)$ – asymptotically faster than sorting, despite "looking similar" to quicksort.

6.3 Amortized analysis: dynamic arrays

A **dynamic array** (Problem 15) starts with some capacity and *doubles* its capacity (copying all existing elements) whenever it becomes full. A single `append` that triggers a resize costs $\Theta(n)$ – but resizes happen only when the size passes $1, 2, 4, 8, \ldots$, so the *total* copying cost over n appends is $1 + 2 + 4 + \cdots + 2^{k-1} < 2 \cdot 2^{k-1} < 2n$, i.e. $O(n)$ total, or $O(1)$ **amortized** per append. This is the standard justification for treating `list.append` in Python (and similarly, `ArrayList`/`Vector` in other languages) as "$O(1)$" despite occasional expensive resizes.

6.4 Priority queues: heap vs. sorted list

A binary min-heap (Problem 12, building on Week 1's heapsort) supports `push` and `pop` in $O(\log n)$, by maintaining the heap property along a single root-to-leaf path. A naive "always keep the list sorted" priority queue (Problem 14) instead pays $O(n)$ per operation (shifting elements to maintain sorted order). Problem 14 asks you to measure this difference directly by counting "element move" operations for both structures on the same sequence of pushes/pops – a concrete $O(n \log n)$ vs. $O(n^2)$ comparison for n insertions followed by n extractions.

Problem 13 extends the heap with a `decrease_key` operation (an **indexed priority queue**), which is exactly the data structure needed for Dijkstra's algorithm, covered in a later week.

6.5 Revisiting Gale–Shapley with arrays

Week 1's Gale–Shapley implementation used Python dictionaries and `.index()` calls to look up preferences and ranks – each `.index()` call is itself $O(n)$, which can silently turn an "$O(n^2)$" algorithm into $O(n^3)$ if you are not careful! Problem 16 re-implements the men-proposing algorithm using only arrays (lists) indexed by integers $0..n-1$, a `collections.deque` for the queue of free men, and a precomputed **rank table** `women_rank[w][m]` so that comparing two suitors is $O(1)$. This makes the true $O(n^2)$ bound from Week 1's Lemma (at most n^2 proposals, each $O(1)$ to process) explicit and verifiable.

6.6 Correctness via induction and invariants

Finally, Problem 17 connects this week's themes back to *correctness* (not just running time): an iterative summation maintains the loop invariant "*total* $= 0+1+\cdots+i$ after processing i", verified at every step with an `assert`; and **binary exponentiation** (repeated squaring) computes a^b in $O(\log b)$ multiplications instead of $O(b)$, by the recurrence

$$a^b = \begin{cases} 1 & b = 0 \\ (a^{b/2})^2 & b \text{ even} \\ (a^{(b-1)/2})^2 \cdot a & b \text{ odd} \end{cases}$$

which is the same "halving recursion" $T(n) = T(n/2) + O(1)$ from Section 3.2, applied to the *exponent* rather than the array size.

7 Summary and Looking Ahead

This week established:

- The formal definitions of O, Ω, Θ, with proofs of transitivity and the polynomial rule.
- The standard hierarchy of growth rates and how to read them off code (loops, nested loops, recursion).
- Polynomial time as a (coarse but useful) definition of tractability, and why the polynomial/exponential divide matters in practice.
- A complete worked example (binary search) analyzed both iteratively (loop invariant + shrinking range) and recursively (recurrence $T(n) = T(n/2) + O(1)$), plus several further examples: max/min, quickselect, amortized analysis of dynamic arrays, heaps vs. sorted lists, and a tighter re-analysis of Gale–Shapley.

Next week, we put this analysis toolkit to use in earnest with **Divide and Conquer** algorithms (Kleinberg & Tardos, Chapter 5): mergesort, Karatsuba multiplication (previewed in Week 1), and the master-theorem-style recurrences introduced informally in Section 3.2 above.